

General-Purpose Input/Output

Chapter Contents

Overview	28
Outside the MCU: Ones and Zeros, Voltages, and Currents	30
Input Signals	30
Output Signals	31
Interfacing with a Switch and LED	31
Inside the MCU	32
Preliminaries: Control Registers and C Code	32
Using CMSIS to Access Hardware Registers with C Code	34
Coding Style for Accessing Bits	34
Reading, Modifying, and Writing Fields in Control Registers	36
Configuring the I/O Path	38
Clock Gating	38
Connecting a Pin to a Peripheral Module	39
GPIO Peripheral	43
GPIO Module Use	44
Putting the C Code Together	46
More Interfacing Examples	46
Driving a Three-Color RGB LED	47
Driving a Speaker	50
Driving the Hot Plate's Heating Element	51
Additional Pin Configuration Options	52
Pull-Ups and Pull-Downs for Inputs	52
Open Drain vs. Push-Pull Outputs	54
Output Drive Strength	54
Configuration Lock	55
Other Options	55
Summary	55
Exercises	55
References	56

Overview

Embedded computers typically need to sense their environment and then control devices in response. Let's start with the basics by learning how to make an embedded computer flash a light and read a switch by using a **general purpose input/output (GPIO) port**.

A microcontroller unit (MCU) contains a central processing unit (CPU) that executes instructions. The CPU is surrounded by specialized hardware circuits called peripherals that interface with external devices or perform other functions for the MCU. Some of these peripherals are integrated into the MCU, whereas others may be added externally on the printed circuit board.

general purpose input/output (GPIO) port

Peripheral with digital input and output bits

In this chapter we introduce the GPIO port. We show how to create a C program to communicate with basic devices such as switches and light-emitting diodes (LEDs) as shown in Figure 2.1. An **input GPIO port bit** lets us read a single bit digital value on an MCU pin. If we connect the

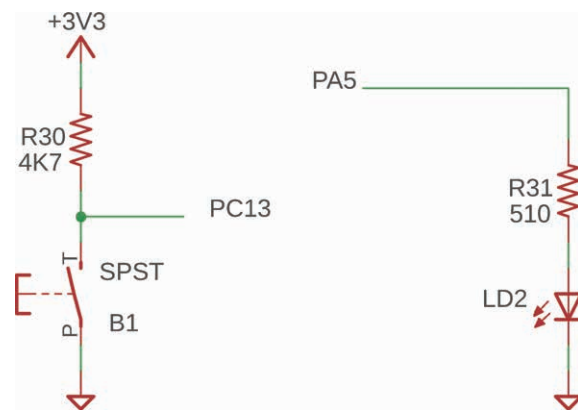


Figure 2.1 A simple digital input and output circuit based on the Nucleo-F091RC board.

pin to a switch, then the program can tell if the switch is open or closed. An **output GPIO port bit** enables the program to set an MCU pin to one of two voltage levels (e.g. either 3.3 V or 0 V). A program can light or extinguish an LED that is connected to this output pin.

input GPIO port bit

Portion of GPIO port which enables program to read a single-bit input signal

output GPIO port bit

Portion of GPIO port which enables program to write a single-bit output signal

The Nucleo-F091RC board has a user input switch B1 and a user output LED LD2 as shown in Figure 2.1. Figure 2.2 shows where to access these signals on the board itself. Throughout this chapter we will follow an example of how to interface a C program with these and other

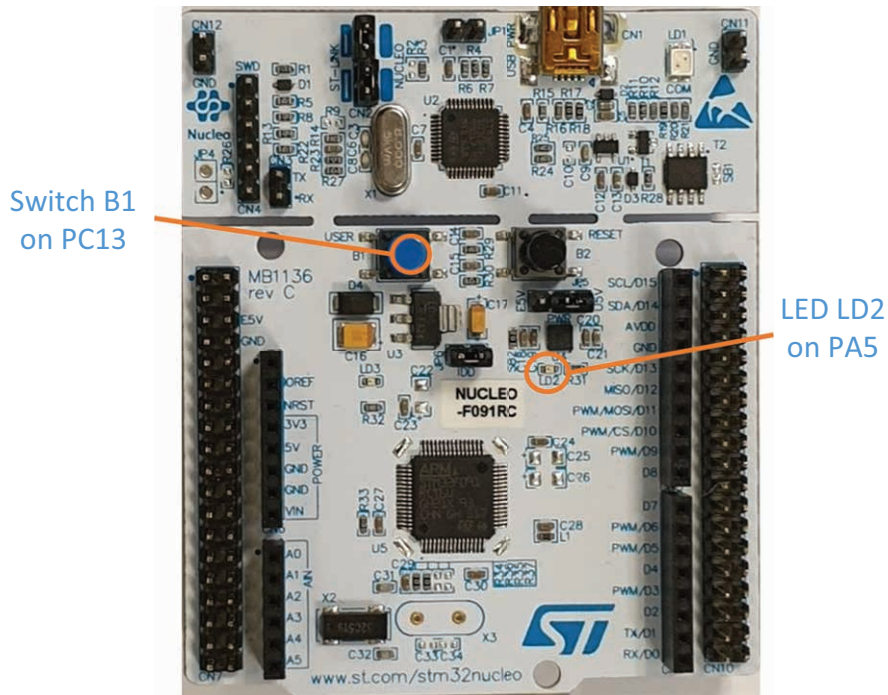


Figure 2.2 Location of switch B1 and LED LD2 on the Nucleo board.

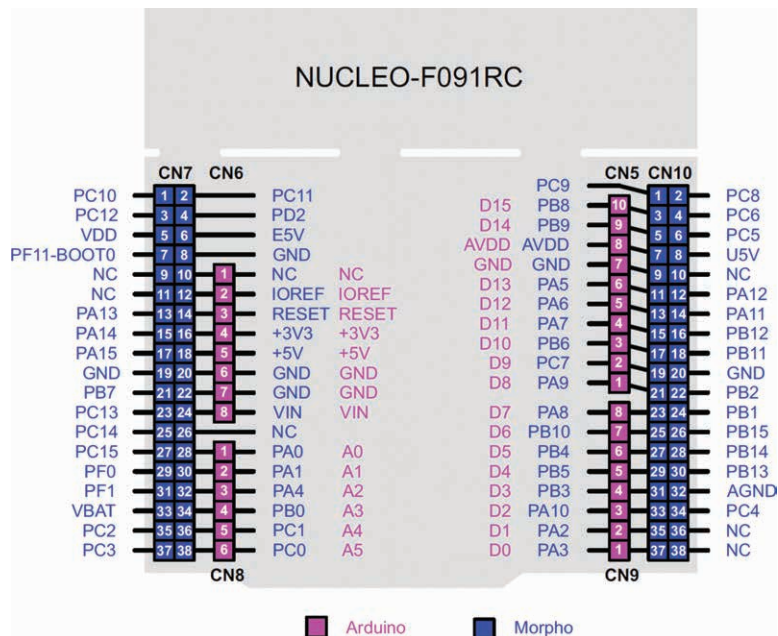


Figure 2.3 Locations for accessing B1 and LD2 signals on Nucleo-F091RC connectors [1]. Signals from other examples in chapter are also shown.

components. Figure 2.3 shows the location of all the GPIO and other signals on the Nucleo board, with the switch and LED signals marked [1].

Some of the implementation details in this chapter are specific to the STM32F0 series of MCUs. For further details, consult the reference manual [2] or data sheet [3].

Outside the MCU: Ones and Zeros, Voltages, and Currents

We will start outside the MCU and work our way in. An MCU is a digital computer, so it operates on ones and zeros. How do we present a one or a zero to an input? What does a one or zero look like on an MCU output?

These values are represented by voltages within specific ranges relative to the MCU's power supply voltage, V_{DD} . The actual voltages do not matter much to an embedded system developer when they are inside the MCU, but they do matter when we want to interface with external devices.

Input Signals

Digital inputs are interpreted based on their voltage levels. How do we supply a one or a zero to a digital input? The supply voltage V_{DD} sets the thresholds for determining whether an input voltage will be considered a one or a zero. For example, the MCU's data sheet might specify that an input voltage between 0 V and $0.3 \times V_{DD}$ will be interpreted as a logic zero, whereas an input voltage between $0.7 \times V_{DD}$ and V_{DD} will be interpreted as a logic one. Figure 2.2 shows how input voltages are interpreted based on the supply voltage V_{DD} . For example, if V_{DD} is 3.3 V, then input values between 0 and 0.99 V will be a logic zero and those between 2.31 and 3.3 V will be a logic one. An input voltage between $0.3 \times V_{DD}$ and $0.7 \times V_{DD}$ is undefined and may be read as a one or a zero. This is useless so we design external circuits to keep the voltage out of that range except when switching.

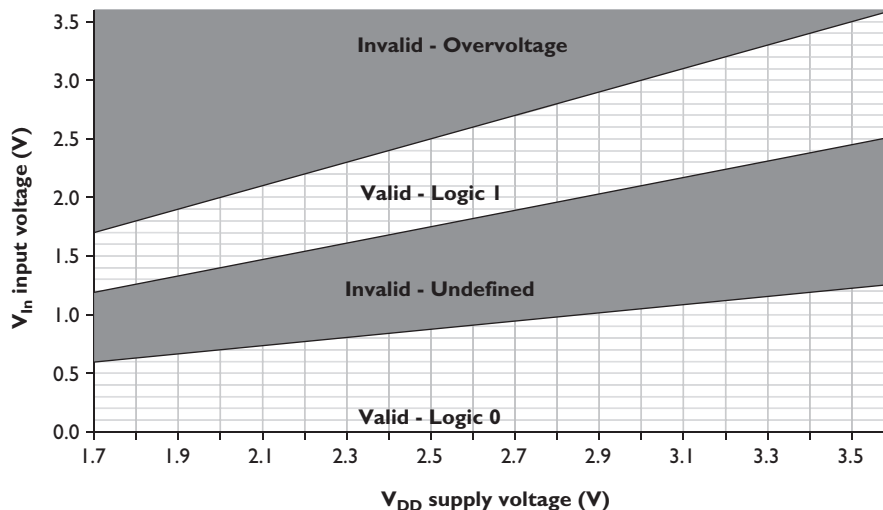


Figure 2.4 Valid input voltage ranges as a function of supply voltage V_{DD} . See I/O port characteristics [3].

Digital inputs do not draw much current from the signal source; they have high input impedance. Typically each GPIO pin will draw no more than 1 μA over the MCU's entire operating temperature range. At room temperature (25°C), the input current will be much smaller (e.g. no more than 25 nA).

Output Signals

Output voltages for digital ports are typically specified as being within a certain voltage range from a supply rail. For this MCU, a normal output generating a logic one will produce a voltage of V_{OH} (out high), which is between V_{DD} and $V_{DD} - 0.4\text{ V}$. A logic zero will produce a voltage of V_{OL} (out low) between 0 and 0.4 V.

This specification only holds true if a limited amount of current is drawn from the output. As that current increases, the output circuit's voltage drop increases, pulling the output voltage away from the supply rails (V_{DD} and ground). If the output circuit draws enough current, it will overload and destroy the output **transistor**. So be careful not to exceed the specified ratings.

For this MCU, the output current I_{OH} or I_{OL} must not exceed 25 mA per pin. Some MCUs include output drivers with more powerful transistors, allowing greater output currents. "High drive" pads may be able to source or sink up more current and still meet the output voltage specifications.

transistor

Basic electronic component which operates as switch or amplifier

The drive current capability also depends on the supply voltage V_{DD} . As V_{DD} falls, the transistors have a higher output resistance, increasing the voltage drop. So at lower operating voltages, the maximum output current available falls. Lowering V_{DD} from 3.3 to 2.0 V might cut I_{OH} and I_{OL} from 5 to 1.5 mA.

Interfacing with a Switch and an LED

Let's consider the switch and LED example shown in Figure 2.1. A switch and a pull-up resistor are connected to signal PC13. When the switch is pressed, the signal is pulled low (logic zero) by the switch; otherwise it is pulled high (logic one) by the resistor. The resistor is large enough that very little current flows when the switch is pressed.

The LED has its **anode** (positive end, base of triangle) connected to an MCU output signal (PA5) through a resistor. The LED's **cathode** (negative end, bar) is connected to ground. An LED's brightness is roughly proportional to the current flowing through it. This current is nearly zero for low voltages, and then rises quickly as the voltage exceeds a threshold. The exact relationship between the voltage and current depends on the type of LED and its temperature. Applying 1.6 V may not light a red LED at all, but 2.0 V will make it bright (with 20 mA of current). Raising the voltage slightly to 2.3 V brightens it more and makes the current shoot up to 43 mA. Raising it further will cause the LED to overheat and fail.

anode*Positive terminal of a polarized component (LED, battery, etc)***cathode***Negative terminal of a polarized component (LED, battery, etc)*

The GPIO port outputs are digital and do not offer such fine-grain voltage control. They can provide two levels: almost ground, and almost V_{DD} (e.g. 3.3 V). The first will correctly keep the LED off, but the second will probably burn out the LED. We need to include resistor R2 to limit the current through the LED and MCU driver output to a safe value.

We can calculate the value R of a current-limiting resistor based on the supply voltage V_{DD} , the LED forward voltage V_F , and the desired LED current I_{LED} :

$$R = \frac{V_{DD} - V_F}{I_{LED}}$$

We start by picking a value of I_{LED} which is safe for both the LED and the MCU output. In this case let's set $I_{LED} = 4$ mA and assume $V_{DD} = 3.0$ V. V_F is 1.8 V for the red LED and 2.7 V for the blue LED. We can now solve for the resistor values. For the red LED, a 300 Ω resistor is needed. For the blue LED, a 75 Ω resistor is needed.

Inside the MCU

Now that we know how to connect these simple devices to the MCU pins, let's look into how to let the program running on the CPU talk with those pins. First the program needs to configure the hardware within the MCU to set up the path between the CPU and the pins. Then the program can read from or write to those pins. Different MCU families use different approaches; here we cover the STM32F0 MCU family.

Preliminaries: Control Registers and C Code

Peripherals are built with **control registers** to allow us to configure them, determine their status and transfer data. Table 2.1 shows some of the control registers for one peripheral module, the Reset and Clock Controller (RCC). These control registers appear as special locations in memory, as indicated by the column marked "Absolute address (hex)". The term "hex" indicates the address is in **hexadecimal** format (base 16). We also can see the width of the register and the value after the MCU is reset (a value of X indicates that the corresponding bits are undefined). The reference manual offers further information [2].